ByteMart Digital

THE COMPLETE GUIDE

Postman

The Complete Guide to API Testing

A complete, hands-on guide to Postman for developers — from sending your first request to writing automated test suites, running them with Newman in CI, mocking APIs, and collaborating in teams.

24	100+	13	6
Chapters	Examples	Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Postman: The Complete Guide

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided "as is".

Trademarks
Postman, Newman, and other product names, logos, and brands referenced are the property of their respective owners and are used for identification purposes only. This guide is an independent educational work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. What Is Postman & Why Use It 4
2. Anatomy of a Request 7
3. HTTP Methods & CRUD 10
4. Reading Responses 12
5. Request Bodies 15
6. Headers, Params & Path Variables 18
7. Authentication & Authorization 21
8. Cookies & Sessions 24
9. File Uploads & Downloads 26
10. Organizing with Collections 28
11. Variables & Scopes 31
12. Environments 34
13. Dynamic Variables & Data Reuse 36
14. Chaining Requests 38
15. The Execution Flow 41
16. Writing Tests with pm.test 43
17. Pre-request Scripts 46
18. The pm Object & Sandbox 49
19. Data-Driven Testing 52
20. Newman: CLI Automation 55
21. CI/CD Integration & Monitoring 58
22. Mock Servers & Documentation 60
23. Team Collaboration & Workspaces 62
24. Best Practices & Workflow 64

MODULE 1 · POSTMAN FUNDAMENTALS

CHAPTER 01

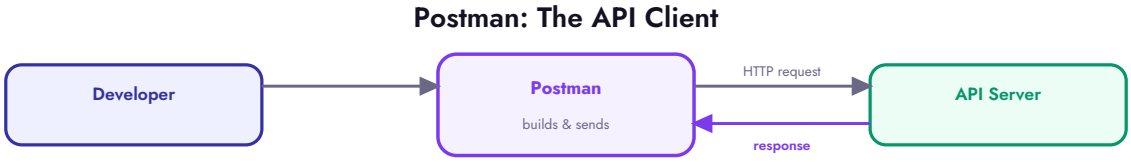
What Is Postman & Why Use It

The API client that replaces ad-hoc testing with a saveable, shareable, automatable workflow.

Every developer who builds or consumes an API needs to send requests and inspect responses. You can do this with curl on the command line — but **Postman** turns that into a visual, organised, and repeatable workflow that a whole team can share.

> **Postman**
> An API platform for building, testing, documenting, and monitoring APIs. At its core it is a client that sends HTTP requests and shows responses — with collections, variables, tests, and automation layered on top.

Where Postman sits



Postman sends HTTP requests to an API and shows the response — no code needed to test.

It replaces ad-hoc curl commands with a saveable, shareable, testable workflow.

Postman builds and sends HTTP requests to an API and shows you the response — no code required.

Why not just use curl

- **Save and organise** requests instead of retyping commands.
- **Reuse values** with variables and environments (dev, staging, prod).
- **Write tests** that assert responses automatically.
- **Automate** whole test suites in CI with Newman.
- **Collaborate** — share collections with your team.

Who uses it

Backend developers test the endpoints they build; frontend developers explore APIs they consume; QA engineers automate API test suites; and teams document and share APIs — all in one tool.

What this guide covers

You will go from sending your first request to writing automated test suites: requests and responses, authentication, collections and variables, scripting and testing, Newman automation, mock servers, and team collaboration — the complete Postman workflow.

© 2026 ByteMart Digital

More than a request sender

Postman is an API client that makes testing saveable, reusable, testable, and shareable. It replaces ad-hoc curl commands with an organised workflow for individuals and teams alike.

MODULE 1 · POSTMAN FUNDAMENTALS

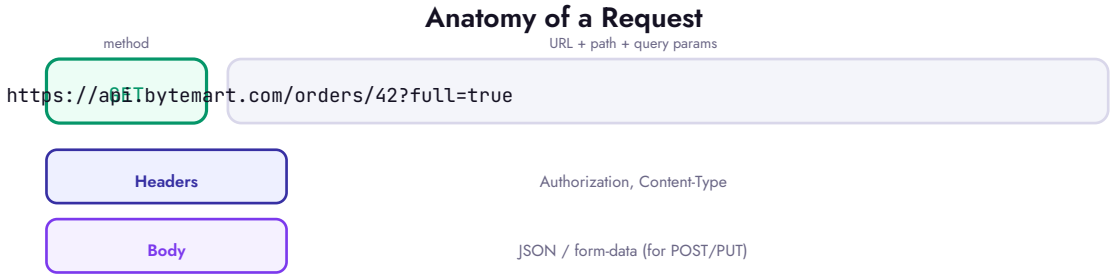
CHAPTER 02

Anatomy of a Request

The four parts of every request, and sending your first call in Postman.

Before anything else, you must understand what a request is. Every HTTP request has the same four parts — and Postman gives you a tab for each.

The four parts



A request is a method, a URL, headers, and (often) a body — the four things you configure.

A request is a method, a URL, headers, and (often) a body — the four things you configure.

Part	What it is
Method	the action: GET, POST, PUT, DELETE
URL	the endpoint, plus path and query params
Headers	metadata: auth, content type
Body	the payload sent (for POST/PUT)

Sending your first request

Create a new request, set the method to GET, enter a URL, and click **Send**. Postman shows the response in the lower pane — status, time, size, headers, and body.

```text
GET  https://jsonplaceholder.typicode.com/todos/1

// click Send -> a JSON response appears below
```

The response pane

The response pane is where you read results: the **status code** and **response time** at the top, and tabs for the **body** (with pretty-printing), **headers**, and **cookies**. This is your feedback loop for every request.

TIP

Postman auto-detects JSON responses and pretty-prints them. Use the **Pretty** / **Raw** / **Preview** tabs to switch how the body is displayed.

Method, URL, headers, body

Every request has four parts, each with its own tab in Postman. Set them, click Send, and read the status, time, headers, and body in the response pane — your core feedback loop.

MODULE 1 · POSTMAN FUNDAMENTALS

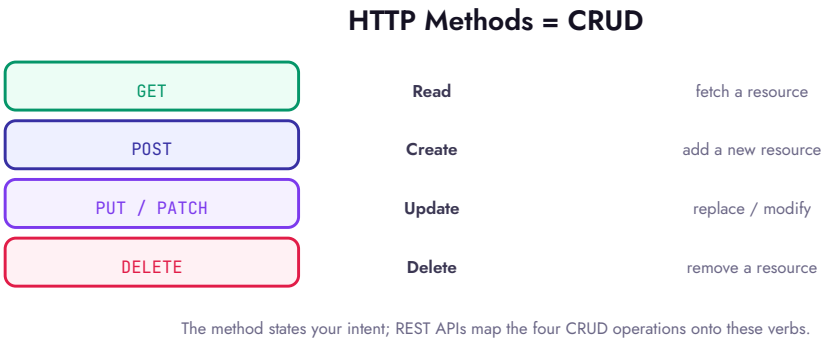
CHAPTER 03

HTTP Methods & CRUD

What each HTTP verb means and how REST APIs map them onto operations.

The HTTP **method** declares your intent. REST APIs map the four basic data operations — create, read, update, delete — onto a small set of verbs, and knowing them is essential to using any API.

The verbs and CRUD



The method states your intent; REST APIs map the four CRUD operations onto these verbs.

The method states your intent; REST maps the four CRUD operations onto these verbs.

Method	Operation	Typical use
GET	Read	fetch a resource or list
POST	Create	add a new resource
PUT	Replace	replace a resource wholesale
PATCH	Update	modify part of a resource
DELETE	Delete	remove a resource

Safe and idempotent

GET is **safe** — it should not change anything. GET, PUT, and DELETE are **idempotent** — repeating them has the same effect as doing them once. POST is neither, which is why refreshing a form-submit can create duplicates.

PUT vs PATCH

PUT replaces the entire resource with what you send; PATCH updates only the fields you provide. Sending a partial body with PUT can accidentally wipe the omitted fields — use PATCH for partial updates.

Verbs express intent

GET reads, POST creates, PUT replaces, PATCH updates, DELETE removes — the CRUD mapping. GET is safe; GET/PUT/DELETE are idempotent; POST is not. Use PATCH, not PUT, for partial updates.

© 2026 ByteMart Digital

MODULE 1 · POSTMAN FUNDAMENTALS

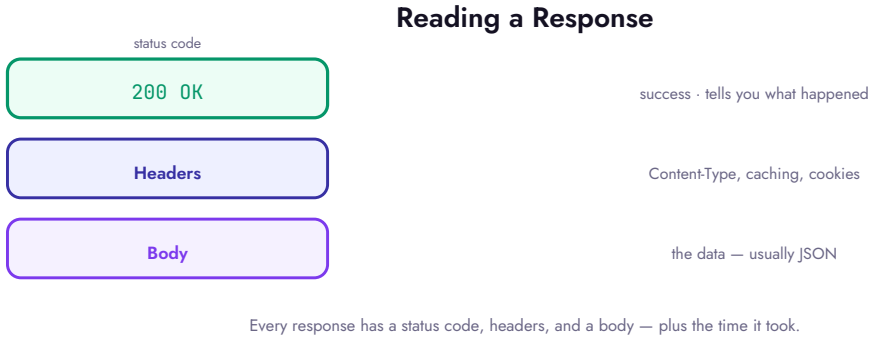
CHAPTER 04

Reading Responses

Status codes, headers, and body — how to interpret what the API sends back.

Sending a request is half the job; understanding the response is the other half. Every response tells you three things: what happened (status), some metadata (headers), and the data (body).

The three parts of a response



Every response has a status code, headers, and a body — plus the time it took.

Status code families

Range	Meaning
2xx	success (200 OK, 201 Created)
3xx	redirection
4xx	client error (400, 401, 404)
5xx	server error (500, 503)

Reading the status

The first digit tells you the category. A `4xx` means you sent something wrong (bad input, missing auth, wrong URL); a `5xx` means the server failed. This distinction points you at where to debug.

Headers and body

Response **headers** carry metadata — `Content-Type`, caching directives, rate-limit info, and cookies. The **body** carries the actual data, usually JSON, which Postman pretty-prints for you.

> **WATCH OUT**
> A `200 OK` does not always mean success at the application level — some APIs return `200` with an error object in the body. Always check the body, not just the status.

Status, headers, body

A response's status code says what happened (2xx success, 4xx your error, 5xx server error), headers carry metadata, and the body carries the data. Read all three — a 200 can still hide an error.

© 2026 ByteMart Digital

MODULE 2 · REQUESTS IN DEPTH

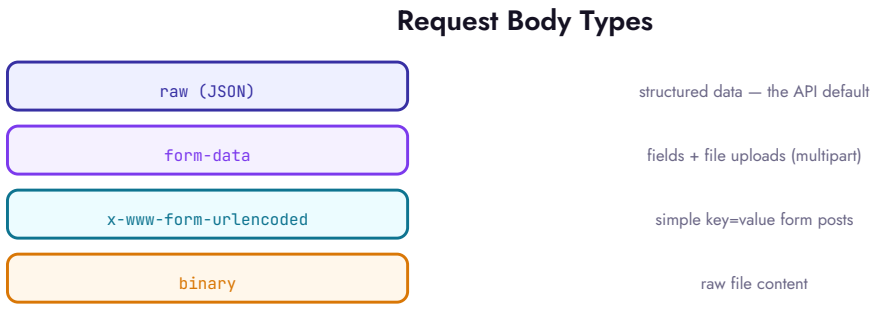
CHAPTER 05

Request Bodies

The body formats Postman supports and when to use each.

For requests that send data — `POST`, `PUT`, `PATCH` — you attach a **body**. Postman's Body tab offers several formats, and choosing the one the API expects is essential.

The body types



Pick the body type the API expects — raw JSON for most REST APIs, form-data for uploads.

Raw JSON — the common case

Most modern REST APIs accept a JSON body. Select **raw** and choose **JSON**, and Postman sets the `Content-Type: application/json` header for you.

```json
{
  "name": "Ada",
  "email": "ada@bytemart.com",
  "role": "admin"
}
```

Form data and uploads

Type	Use for
form-data	form fields plus file uploads
x-www-form-urlencoded	simple key=value form posts
binary	sending a single raw file
GraphQL	GraphQL queries and variables

WATCH OUT

The body type must match the `Content-Type` the server expects. Sending JSON to an endpoint that wants form-encoded data (or vice versa) causes confusing `400` or `415` errors.

Content-Type is set for you

When you pick a body type, Postman adds the matching `Content-Type` header automatically. If you override it manually, make sure it still matches the body — a mismatch is a common source of bugs.

Match body to what the API wants

Requests that send data use a body: raw JSON for most REST APIs, form-data for uploads, urlencoded for simple forms. Postman sets the matching Content-Type — keep the two in sync.

MODULE 2 · REQUESTS IN DEPTH

CHAPTER 06

Headers, Params & Path Variables

The three ways data travels in a request — headers, query params, and path variables.

Beyond the body, a request carries data in three other places: **headers**, **query parameters**, and **path variables**. Each has a distinct purpose, and Postman makes all three easy to manage.

Headers

Headers carry metadata about the request — authentication, content type, accepted formats, and custom values. Postman shows auto-generated headers and lets you add your own in the Headers tab.

```text
Authorization: Bearer eyJhbGci...
Content-Type: application/json
Accept: application/json
```

Query parameters

Query params come after a `?` in the URL as key-value pairs, used for filtering, sorting, and pagination. Postman's Params tab keeps them in sync with the URL as you edit either one.

```text
GET /orders?status=paid&page=2&sort=-createdAt
```

Path variables

Path variables are part of the URL path itself, identifying a specific resource. In Postman, a `:id` segment becomes an editable path variable.

```text
GET /orders/:id      // :id is a path variable
GET /orders/42       // resolved
```

Which goes where

Location	Carries
Header	metadata (auth, content type)
Query param	filters, options, pagination
Path variable	which specific resource

© 2026 ByteMart Digital

Three places for request data

Headers carry metadata like auth and content type; query params (after ?) filter and paginate; path variables (:id) identify a specific resource. Postman keeps params in sync with the URL.

© 2026 ByteMart Digital

MODULE 2 · REQUESTS IN DEPTH

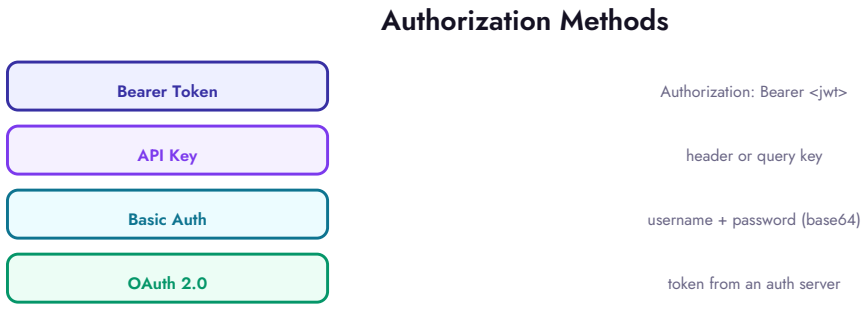
CHAPTER 07

Authentication & Authorization

Configuring the common auth schemes so protected endpoints accept your requests.

Most real APIs are protected. Postman's **Authorization** tab builds the correct credentials for you — no manual header crafting — across every common scheme.

The common schemes



Postman's Authorization tab builds the right header and can inherit it from a collection.

Type	How it works
Bearer Token	sends Authorization: Bearer <token>
API Key	a key in a header or query param
Basic Auth	base64 username:password
OAuth 2.0	obtains a token via a flow

Bearer tokens — the JWT case

For token-based APIs, choose **Bearer Token** and paste (or reference) the token. Postman adds the `Authorization` header. Better still, store the token in a variable so you never hard-code it.

```text
// Authorization tab -> Bearer Token
Token: {{access_token}}
// sends -> Authorization: Bearer eyJhbGci...
```

OAuth 2.0

Postman can run an OAuth 2.0 flow for you — you configure the auth URL, token URL, client id, and scopes, and it obtains an access token you can reuse across requests. This automates the login step of the Authorization Code flow.

Inherit auth from the collection

TIP

Set auth once at the **collection** level and let each request **inherit** it. Change the token in one place instead of on every request — the single biggest time-saver for protected APIs.

Let Postman build the credentials

The Authorization tab handles Bearer, API Key, Basic, and OAuth 2.0, adding the right header for you. Store tokens in variables and set auth at the collection level so requests inherit it.

MODULE 2 · REQUESTS IN DEPTH

CHAPTER 08

Cookies & Sessions

How Postman handles cookie-based authentication and stateful APIs.

Not every API uses tokens — many web applications use **cookies** and server-side sessions. Postman manages cookies automatically, mirroring how a browser behaves.

Automatic cookie handling

When a response sets a cookie (via `Set-Cookie`), Postman stores it and sends it back on subsequent requests to the same domain — just like a browser. This makes session-based login work without manual effort.

The cookie manager

Postman's **Cookie Manager** lets you view, add, edit, and delete cookies per domain. This is invaluable for debugging session issues — you can inspect exactly what is being sent and clear a stale session.

Session login flow

```text
1. POST /login  ->  response sets a session cookie
2. Postman stores the cookie automatically
3. GET /profile  ->  cookie sent, you are logged in
```

Tokens vs cookies

	Cookies	Tokens
Storage	browser / Postman	you manage
Sent	automatically	in a header
Typical use	web sessions	REST APIs, SPAs

NOTE

Cookie handling is tied to the domain. If your API and auth run on different domains, cookies may not carry across — a common source of 'logged in but unauthorised' confusion.

Cookies handled like a browser

Postman automatically stores and resends cookies per domain, so session-based logins just work. The Cookie Manager lets you inspect and clear them — essential for debugging stateful APIs.

MODULE 2 · REQUESTS IN DEPTH

CHAPTER 09

File Uploads & Downloads

Sending files to an API and handling binary responses.

APIs often deal with files — profile pictures, documents, exports. Postman handles both directions: uploading files in a request and receiving binary data in a response.

Uploading a file

To upload, select the **form-data** body type, change a field's type from **Text** to **File**, and choose the file. Postman sends it as a multipart request — the standard way to upload.

```text
Body -> form-data
  file  [File]  -> profile.png
  name  [Text]  -> "avatar"
// sent as multipart/form-data
```

Multiple files and fields

You can mix file and text fields freely in one form-data body, and include several file fields — useful for endpoints that accept a document plus metadata together.

Downloading a response

When a response is a file (a PDF, image, or export), Postman shows a preview where possible and offers **Save Response to file** to download it. Binary responses are handled without corrupting the bytes.

WATCH OUT

Files referenced in a request are stored by **path**, not embedded in the collection. If you share a collection, the file paths will not resolve on a teammate's machine — note this when collaborating.

Files both ways

Upload files with the form-data body type by setting a field to File (sent as multipart); download binary responses with Save Response to file. Remember file paths are local, not shared with the collection.

© 2026 ByteMart Digital

MODULE 3 · COLLECTIONS & VARIABLES

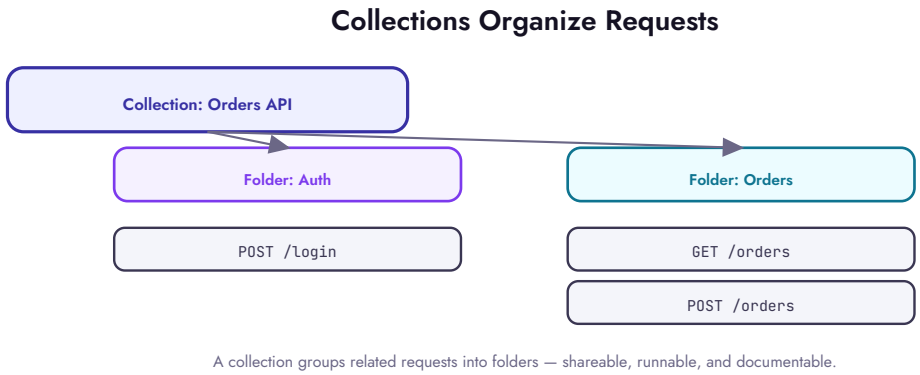
CHAPTER 10

Organizing with Collections

Grouping related requests into a structured, shareable, runnable unit.

A handful of loose requests quickly becomes unmanageable. **Collections** are Postman's core organising unit — they group related requests into folders, and unlock sharing, running, and documentation.

The structure



A collection groups related requests into folders — shareable, runnable, and documentable.

A collection contains folders, and folders contain requests. You might have a collection per API, with folders for each resource (Auth, Orders, Products) and the requests inside each.

What collections unlock

- **Run** the whole collection at once with the Collection Runner.
- **Share** it with teammates or export it as a file.
- **Document** it — Postman generates docs from the collection.
- **Automate** it in CI with Newman.
- **Attach** collection-level auth, variables, and scripts.

Collection-level settings

A collection can hold shared **auth** (inherited by every request), shared **variables**, and shared **pre-request/test scripts** that run for every request. Setting these once keeps your requests DRY.

> **TIP**
> Structure a collection to tell a story: put a login request first, then the requests that depend on it. A well-ordered collection doubles as living documentation and an end-to-end test.

© 2026 ByteMart Digital

The unit of organisation

Collections group requests into folders and unlock running, sharing, documentation, and automation. Collection-level auth, variables, and scripts apply to every request — organise once, reuse everywhere.

© 2026 ByteMart Digital

MODULE 3 · COLLECTIONS & VARIABLES

CHAPTER 11

Variables & Scopes

Replacing hard-coded values with variables, and understanding which scope wins.

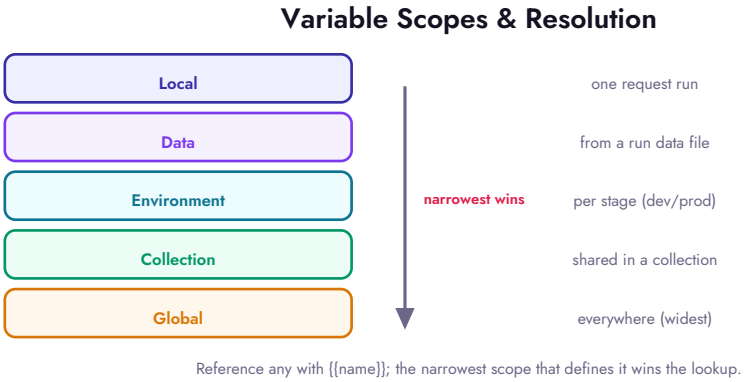
Hard-coding URLs, tokens, and ids into every request is fragile and repetitive. **Variables** — referenced with `{{name}}` — replace them, and Postman's **scopes** decide which value applies where.

Using a variable

```text
// instead of hard-coding the host everywhere:
GET {{base_url}}/orders/{{order_id}}

// change base_url once -> every request updates
```

The scopes



Reference any variable with {{name}}; the narrowest scope that defines it wins the lookup.

Scope	Lives	Use for
Global	everywhere	rare, app-wide values
Collection	one collection	shared config
Environment	the active env	per-stage values
Local / Data	one run	temporary values

Resolution order

When a variable is defined in more than one scope, the **narrowest wins**: local and data override environment, which overrides collection, which overrides global. This lets a general default be overridden for a specific run.

WATCH OUT

Never hard-code secrets or tokens into requests, and be careful sharing collections — values stored in the collection or globals can leak. Keep secrets in environments and avoid exporting them.

Variables everywhere, narrowest wins

Reference values with {{name}} instead of hard-coding them. Scopes range from global (widest) to local/data (narrowest), and the narrowest defined scope wins — letting you override defaults per stage or run.

© 2026 ByteMart Digital

MODULE 3 · COLLECTIONS & VARIABLES

CHAPTER 12

Environments

Switching between dev, staging, and production with a single dropdown.

The same collection usually runs against several backends — your local machine, staging, production. **Environments** are named sets of variables you switch between, so one collection works everywhere.

An environment is a variable set

Each environment defines the same variable names with different values. A **dev** environment points `base_url` at localhost; a **prod** environment points it at the live API. Switch environments and every request follows.

```text
Environment: Dev          Environment: Prod
  base_url = localhost:8080   base_url = api.bytemart.com
  api_key  = dev-key-123      api_key  = prod-key-xyz
```

Switching contexts

The environment dropdown in the top-right selects the active environment. All `{{variables}}` resolve against it — so running the exact same requests against staging or production is a one-click change.

Initial vs current values

Postman distinguishes an **initial value** (shared when you export) from a **current value** (local to you, never shared). Put secrets only in the current value so they are not exported with the environment.

WATCH OUT

Store real secrets in **current values**, not initial values. Initial values are shared on export — a token left in an initial value can be leaked to anyone you share the environment with.

One collection, many targets

Combined with variables, environments are what make a collection portable: write it once, then point it at any backend by switching environments. This is the foundation for testing across stages and for CI.

Switch backends in one click

Environments are named variable sets (dev, staging, prod) you switch via a dropdown, so one collection runs anywhere. Keep secrets in current values — never initial values — to avoid leaking them on export.

MODULE 3 · COLLECTIONS & VARIABLES

CHAPTER 13

Dynamic Variables & Data Reuse

Generating fresh test data on the fly with built-in dynamic variables.

Hard-coded test data goes stale and causes collisions — creating the same user twice fails. Postman's **dynamic variables** generate fresh values every run, so your tests stay repeatable.

Built-in dynamic variables

Dynamic variables start with $ and produce a new value each time they are used — no setup required. They are perfect for creating unique test data.

Variable	Generates
{{$guid}}	a unique UUID
{{$timestamp}}	the current Unix time
{{$randomInt}}	a random integer
{{$randomEmail}}	a fake email address
{{$randomFullName}}	a fake name

Using them in a body

```json
{
  "id": "{{$guid}}",
  "email": "{{$randomEmail}}",
  "createdAt": {{$timestamp}}
}
```

Why this matters for testing

A test that creates a user with a fixed email works once, then fails on the second run because the email already exists. A dynamic `{{$randomEmail}}` sidesteps this — every run gets a fresh, non-colliding value.

TIP

Combine dynamic variables with a pre-request script to build more complex data — for example, generate a value once and store it in a variable so several requests share the same one.

Fresh data every run

Dynamic variables like {{$guid}}, {{$timestamp}}, and {{$randomEmail}} generate new values on each use, keeping tests repeatable and collision-free — no more 'already exists' failures from hard-coded data.

MODULE 3 · COLLECTIONS & VARIABLES

CHAPTER 14

Chaining Requests

Passing data from one response into the next request to build real workflows.

Real API workflows are sequences: log in, then use the returned token; create an order, then fetch it by its new id. **Chaining** passes data from one response into the next request — the key to end-to-end testing.

The pattern



A test script saves a value from one response into a variable; the next request reads it.

Save in a test script

In the first request's **Tests** (post-response) script, read a value from the response and store it in a variable with `pm.environment.set` (or `pm.collectionVariables.set`).

```javascript
// POST /login  -> Tests tab
const body = pm.response.json();
pm.environment.set("access_token", body.token);
```

Read in the next request

The next request references the saved variable with `{{access_token}}` in its header, URL, or body — login once, then call every protected endpoint without re-entering the token.

```text
GET {{base_url}}/orders
Authorization: Bearer {{access_token}}
```

Chaining ids

The same technique threads created resources through a flow: save the `id` from a `POST` response, then use `{{order_id}}` in the following `GET`, `PATCH`, or `DELETE` — a complete lifecycle test.

© 2026 ByteMart Digital

Thread data through requests

Chaining saves a value from one response into a variable (in a test script) that the next request reads. It powers real workflows — log in then call protected endpoints, or create then fetch a resource.

© 2026 ByteMart Digital

MODULE 4 · SCRIPTING & TESTING

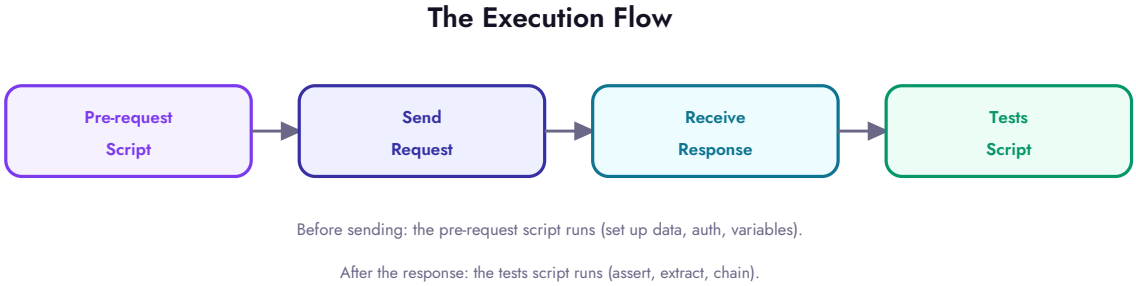
CHAPTER 15

The Execution Flow

When pre-request and test scripts run, and what each is for.

Postman is not just a request sender — it runs your JavaScript before and after each request. Understanding **when** each script runs unlocks setup, assertions, and automation.

The order of execution



The pre-request script runs before sending; the tests script runs after the response arrives.

Stage	Runs	Used for
Pre-request	before sending	setup, data, auth
(request sent)	—	—
Tests	after response	assert, extract, chain

Pre-request scripts

The **pre-request** script runs just before the request is sent. Use it to generate timestamps, compute a signature, fetch a fresh token, or set up variables the request depends on.

Test scripts

The **tests** (post-response) script runs after the response arrives. Use it to assert the response is correct, extract values for chaining, and log information — this is where automated testing lives.

Scripts at every level

Scripts can live on a single request, on a folder, or on the whole collection. Collection- and folder-level scripts run for every request inside — ideal for shared setup like refreshing auth before each call.

Before and after each request
The pre-request script runs before sending (setup, data, auth); the tests script runs after the response (assert, extract, chain). Both can live at request, folder, or collection level for shared logic.

MODULE 4 · SCRIPTING & TESTING

CHAPTER 16

Writing Tests with pm.test

Turning manual checks into automated assertions that pass or fail.

The heart of API testing in Postman is the `pm.test` function. Each test has a name and a function of assertions; Postman reports each as pass or fail after every run.

A basic test

```javascript
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});
```

Asserting on the body

Parse the JSON body with `pm.response.json()`, then assert on its fields using `pm.expect` (a Chai-style assertion library built in).

```javascript
pm.test("Returns the right order", function () {
    const json = pm.response.json();
    pm.expect(json.id).to.eql(42);
    pm.expect(json.status).to.eql("paid");
    pm.expect(json.items).to.be.an("array").that.is.not.empty;
});
```

Common assertions

Assertion	Checks
to.have.status(200)	the status code
to.have.header("...")	a response header exists
pm.expect(x).to.eql(y)	a value equals y
responseTime below N	the response was fast enough

Multiple tests per request

Write as many `pm.test` blocks as you like on one request — each is reported separately, so a partial failure still tells you exactly which check broke.

© 2026 ByteMart Digital

Assertions that pass or fail

pm.test(name, fn) defines a named check; inside, pm.response.to.have.status() and pm.expect() (Chai) assert on status, headers, and body. Multiple tests per request pinpoint exactly what broke.

MODULE 4 · SCRIPTING & TESTING

CHAPTER 17

Pre-request Scripts

Preparing everything a request needs before it is sent.

Sometimes a request needs work done before it goes out — a fresh timestamp, a computed value, or a token. The **pre-request script** is where that setup happens.

Setting variables

Compute a value and store it in a variable the request will use. A common case is generating a timestamp or nonce required by the API.

```javascript
// pre-request script
const now = Date.now();
pm.variables.set("requestTime", now);
// request body can then use {{requestTime}}
```

Fetching a token before the request

The pre-request script can send its own request with `pm.sendRequest` — for example to fetch a fresh access token — and store the result before the main request runs. This automates auth so you never manually refresh tokens.

```javascript
pm.sendRequest({
    url: pm.environment.get("base_url") + "/login",
    method: "POST",
    header: { "Content-Type": "application/json" },
    body: { mode: "raw", raw: JSON.stringify({ user: "svc", pass: "x" }) }
}, function (err, res) {
    pm.environment.set("access_token", res.json().token);
});
```

Conditional setup

Because it is JavaScript, a pre-request script can branch — only fetch a new token if the stored one is missing or expired, for instance — keeping runs fast and correct.

> **TIP**
> Put a token-refresh pre-request script at the **collection** level so every request gets valid auth automatically, without repeating the logic on each one.

Set up before you send

The pre-request script runs before the request, preparing variables, timestamps, and even fetching tokens with pm.sendRequest. Placed at collection level, it can auto-refresh auth for every request.

MODULE 4 · SCRIPTING & TESTING

CHAPTER 18

The pm Object & Sandbox

The JavaScript API that scripts use to read requests, responses, and variables.

All Postman scripting goes through the `pm` object — the interface to the request, the response, variables, and assertions. Knowing its main parts is knowing how to script anything.

The main areas

pm.*	Gives you
pm.request	the outgoing request
pm.response	the response (json, code, headers)
pm.environment	get/set environment variables
pm.collectionVariables	get/set collection variables
pm.variables	resolve any variable
pm.test / pm.expect	define tests and assertions

Reading the response

```javascript
pm.response.code;             // 200
pm.response.responseTime;     // ms
pm.response.json();           // parsed body
pm.response.headers.get("Content-Type");
```

Getting and setting variables

```javascript
const base = pm.environment.get("base_url");
pm.collectionVariables.set("order_id", 42);
pm.environment.unset("temp_token");   // remove
```

The sandbox

Scripts run in a JavaScript **sandbox** with common libraries available (including Chai for assertions and Lodash for utilities). It is standard JavaScript, so loops, conditionals, and functions all work as expected.

© 2026 ByteMart Digital

One object for everything

The pm object exposes pm.request, pm.response, pm.environment, pm.collectionVariables, and pm.test/pm.expect. Scripts run in a JavaScript sandbox with Chai and Lodash — read responses, manage variables, and assert.

MODULE 4 · SCRIPTING & TESTING

CHAPTER 19

Data-Driven Testing

Running a collection many times with different inputs from a data file.

To test an endpoint properly you need many inputs — valid, invalid, and edge cases. The **Collection Runner** with a **data file** runs your collection once per row, injecting each row's values.

How it works



The runner executes a collection once per data row, injecting each row's values.

The data file

Provide a CSV or JSON file where each row is one test case. During the run, each column becomes a variable you reference with `{{column}}` (or read via `pm.iterationData`).

```text
email,password,expectedStatus
ada@x.com,secret1,200
bad-email,secret1,400
ada@x.com,wrong,401
```

Referencing the data

```javascript
// request body uses {{email}} and {{password}}
// the test asserts against the expected status:
pm.test("status matches case", function () {
    const expected = Number(pm.iterationData.get("expectedStatus"));
    pm.expect(pm.response.code).to.eql(expected);
});
```

One collection, many cases

The runner iterates over every row, running the collection and its tests each time, and reports pass/fail per iteration. One collection plus a data file becomes a full test matrix — the foundation of thorough API testing.

© 2026 ByteMart Digital

Test a matrix from a file

The Collection Runner runs a collection once per data-file row, exposing each column as a variable. Reference it with {{column}} or pm.iterationData — turning one collection into a full suite of valid, invalid, and edge cases.

MODULE 5 · AUTOMATION & COLLABORATION

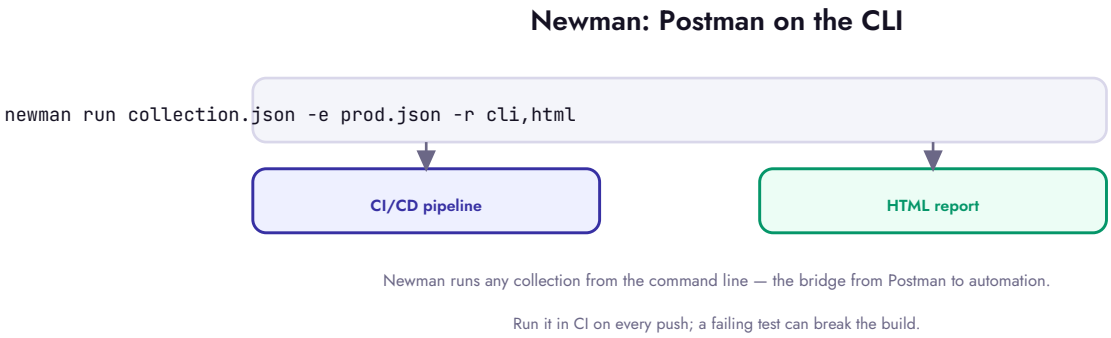
CHAPTER 20

Newman: CLI Automation

Running your collections from the command line — the bridge to automation.

Everything you build in the Postman app can run headless from the command line with **Newman**, Postman's CLI runner. This is the bridge from interactive testing to true automation.

Installing and running



Newman runs any collection from the command line — the bridge from Postman to automation.

```bash
npm install -g newman

newman run OrdersAPI.postman_collection.json \
  -e prod.postman_environment.json \
  -d cases.csv \
  -r cli,html
```

The key options

Option	Does
-e	use an environment file
-d	use a data file (data-driven run)
-r	reporters (cli, html, json, junit)
–bail	stop on the first failure

Exit codes drive automation

Newman exits with a non-zero code if any test fails. That single fact is what lets it gate a pipeline — a failing API test can fail the build, just like a failing unit test.

Reports

The `-r html` reporter produces a shareable report of every request and assertion — useful as a CI artifact your team can inspect after each run.

> **Postman on the command line**
> Newman runs collections headless: newman run collection -e env -d data -r cli,html. It supports environments, data files, and reporters, and its non-zero exit on failure lets it gate a CI pipeline.

MODULE 5 · AUTOMATION & COLLABORATION

CHAPTER 21

CI/CD Integration & Monitoring

Running API tests automatically on every change, and watching production continuously.

Automated tests deliver the most value when they run automatically. Newman slots into any CI pipeline, and Postman monitors watch your live API on a schedule.

Newman in CI

Add a step that installs Newman and runs your collection. Because Newman fails the build on any test failure, a broken API is caught before it ships — the same safety net as unit tests, at the API level.

```yaml
# GitHub Actions step
- name: API tests
  run: |
    npm install -g newman
    newman run collection.json -e ci.json --bail
```

Where to run it

- **On every pull request** — catch regressions before merge.
- **After deploy** — smoke-test the freshly deployed environment.
- **On a schedule** — detect drift and outages.

Monitors

A Postman **monitor** runs a collection on a schedule from Postman's cloud — every few minutes or hours — and alerts you when tests fail or latency climbs. This turns your test collection into uptime and correctness monitoring for production.

Test suite as living contract

Run the same collection in development, in CI, and as a monitor, and it becomes a single source of truth for how the API should behave — authored once, used everywhere.

> **Automate everywhere**
> Newman runs in CI to gate merges and smoke-test deploys, failing the build on any test failure. Postman monitors run collections on a schedule from the cloud, turning your tests into production monitoring.

© 2026 ByteMart Digital

MODULE 5 · AUTOMATION & COLLABORATION

CHAPTER 22

Mock Servers & Documentation

Building against an API before it exists, and generating docs from your collection.

Two Postman features accelerate teams beyond testing: **mock servers** let clients build before the real API is ready, and **documentation** is generated straight from your collection.

Mock servers



A mock server returns saved example responses without a real backend.

A mock server returns **saved example responses** from your collection at a real URL — no backend required. Frontend and client teams can develop against the agreed contract while the API is still being built.

How mocks work

You add **examples** to your requests — sample request-and-response pairs — and Postman serves them from the mock URL. Different examples can respond to different inputs, simulating real behaviour closely enough to build against.

Documentation from the collection

Postman generates API **documentation** automatically from a collection — descriptions, parameters, example requests, and responses. Because it is generated from the same requests you test, the docs stay in sync with reality.

- Write descriptions on the collection, folders, and requests.
- Add examples so the docs show real request and response bodies.
- Publish the docs to a shareable public or private URL.

> **Mock early, document automatically**
> Mock servers serve saved example responses at a real URL so clients build before the backend exists. Documentation is generated from the collection, staying in sync with the requests you actually test.

© 2026 ByteMart Digital

MODULE 6 · MASTERY

CHAPTER 23

Team Collaboration & Workspaces

Sharing work through workspaces, and keeping collections in sync across a team.

Postman is built for teams, not just individuals. **Workspaces** organise shared work, and Postman's syncing keeps everyone on the same collections and environments.

Types of workspace

Workspace	For
Personal	your own work, private
Team	shared with your organisation
Public	shared with the world

Sharing collections

In a team workspace, collections and environments are shared live — a change one person makes is visible to all. This replaces emailing exported JSON files back and forth, which quickly drift out of sync.

Forking and merging

Postman supports a Git-like model: **fork** a collection to work on it in isolation, then create a **pull request** to merge your changes back. This lets teams review changes to shared collections rather than overwriting each other.

Version safety

WATCH OUT

In a shared workspace, everyone edits the same collection. Use forks and pull requests for significant changes, and be mindful that secrets in shared environments are visible to the whole team — keep them in personal current values.

Comments and roles

Team members can comment on requests, and roles control who can edit versus view — so a large team can collaborate on APIs without stepping on each other.

Built for teams

Workspaces (personal, team, public) organise shared work, with collections and environments synced live. Fork-and-pull-request workflows let teams review changes safely — and keep shared secrets out of shared scopes.

© 2026 ByteMart Digital

MODULE 6 · MASTERY

CHAPTER 24

Best Practices & Workflow

The habits that turn Postman from a request sender into a professional API workflow.

You now know Postman end to end. This closing chapter distils the practices that keep your collections maintainable, your tests trustworthy, and your secrets safe.

Organisation

- **One collection per API**, with folders per resource.
- **Order requests logically** — login first, then dependent calls.
- **Describe** collections, folders, and requests for living docs.

Variables and secrets

- **Never hard-code** URLs, tokens, or ids — use variables.
- **Environments per stage** (dev, staging, prod).
- **Keep secrets in current values**, never initial values or the collection.
- **Inherit auth** from the collection instead of per request.

Testing

- **Write tests**, not just manual checks — assert status and body.
- **Chain requests** to cover real workflows end to end.
- **Data-drive** with files to cover valid, invalid, and edge cases.
- **Run in CI** with Newman so regressions fail the build.

The professional workflow

The full loop: design and organise a collection, parameterise it with variables and environments, add tests and chaining, run it locally, automate it in CI with Newman, monitor production, and share it in a team workspace. That is Postman used to its full potential — not a toy for one-off requests, but the backbone of an API workflow.

> **From requests to a workflow**
> Organise one collection per API, parameterise with variables and per-stage environments, keep secrets safe, write real tests, chain and data-drive them, automate with Newman in CI, and collaborate in workspaces.

© 2026 ByteMart Digital